

Shortest paths: Dijkstra's algorithm

Given a graph and a source vertex, Dijkstra's algorithm finds the shortest path from the source vertex to each other vertex in the graph.

Definition

Given:

- *source*, the vertex to measure shortest path distance from.
- $distance[u, v]$, the length of the edge connecting vertex u to vertex v . If there is no such edge, then $distance[u, v] \leftarrow \infty$.

Let:

- Q be the set of nodes for which we have yet to find the shortest path.
- $distance[u]$ be the length of the shortest path from *source* to u . We initialize $distance[u] \leftarrow \infty$ if $u \neq source$ and $distance[source] \leftarrow 0$.
- $previous[u]$ be the vertex that is u 's predecessor in the shortest path from *source* to u . That is, the shortest path can be described as:

$$u \Leftarrow previous[u] \Leftarrow previous[previous[u]] \Leftarrow \dots \Leftarrow source$$

We set $previous[u] \leftarrow \text{null}$ if we have yet to find the shortest path from *source* to u (i.e. if $u \in Q$).

Dijkstra's algorithm

```
1  for each vertex  $v$ :
2       $distance[v] \leftarrow \infty$ 
3       $previous[v] \leftarrow \text{null}$ 
4   $distance[source] \leftarrow 0$ 
5   $Q \leftarrow$  set of all vertices in the graph

6  while  $Q$  is not empty:
7       $u \leftarrow$  vertex  $v$  in  $Q$  with smallest  $distance[v]$ 
8      remove  $u$  from  $Q$ 
9      if  $distance[u] = \infty$ : break
10     for each neighbor  $v$  of  $u$ :
11         if  $distance[u] + distance[u, v] < distance[v]$ :
12              $distance[v] \leftarrow distance[u] + distance[u, v]$ 
13              $previous[v] \leftarrow u$ 
14  return  $distance$ 
```

Analysis

Running time is a bit difficult to analyse, and will depend on what data structures you use. In particular, there are two choices to make:

- To implement Q as an array (or list) or as a heap. If it is implemented as an array (or list), finding the smallest element will take linear time $O(|V|)$ and removing the element will take constant time. If it is implemented as a heap, finding the smallest element will take constant time and removing it will take logarithmic time $O(\log |V|)$. The heap offers better running times however is more difficult to implement and requires some maintenance whenever *distance* values are updated in the inner loop (line 12). For small problems it is often sufficient to use an array (or list) whereas heaps should be kept for the larger, more time-constrained problems.
- To implement the graph as an adjacency list or adjacency matrix. If implemented as a matrix, the inner loop (lines 10-13) will iterate over every vertex each time it is run $O(|V|)$. If it is implemented as an adjacency list, the situation is a bit more complicated. Since the outer loop runs at most once for each vertex, the inner loop (over the life of the program) will run at most once for each *edge* in the graph (twice for each edge if graph is undirected)

Handwaving a bit: the algorithm will take $O(|V|^2)$ if arrays (or lists) are used to implement Q regardless of how the graph is implemented. On the other hand, if a heap is used to implement Q , then the algorithm will take $O(|V|^2 + |V| \log |V|) = O(|V|^2)$ if an adjacency matrix is used or $O((|V| + |E|) \log |V|)$ if adjacency lists are used. In general, adjacency lists and heaps offer the fastest running time (especially for sparse graphs), while adjacency matrices and arrays (or lists) are easier to implement.

All-pairs shortest paths: Floyd-Warshall algorithm

Given an graph represented as an adjacency matrix, Floyd-Warshall finds the distance of the shortest path between each pair of vertices.

Definition:

Floyd-Warshall can be phrased as a dynamic programming problem: What is the shortest path between two vertices using only the first k vertices in the graph? When $k = 0$, the shortest paths are obviously just the edge lengths between neighboring vertices. Going to $k = 1$, we then consider paths such as $u \Rightarrow 1 \Rightarrow v$ and so on.

Given:

- n , the number of vertices.
- $distance[u, v]$, the length of the edge connecting vertex u to vertex v . If there is no such edge, then we initialize $distance[u, v] \leftarrow \infty$. After the algorithm has run, $distance[u, v]$ will be the length of the shortest path from u to v or ∞ if u is not connected to v .

Floyd-Warshall

```
1 for  $middle \leftarrow 1 \dots n$ 
2   for  $start \leftarrow 1 \dots n$ 
3     for  $end \leftarrow 1 \dots n$ 
4        $distance[start, end] \leftarrow \min(distance[start, end],$   
                                          $distance[start, middle] + distance[middle, end])$ 
```

Note: The order of the loops is important! Specifically, the outermost loop must iterate over the “*middle*” index. You can swap the two innermost loops though.

Analysis

There are three nested loops, each of which iterates over all vertices. Thus the time complexity is $O(n^3)$. The path distance is stored in an $n \times n$ matrix and so the space complexity is $O(n^2)$.

Eulerian paths

A Eulerian path is a path through a graph that uses each edge once. A Eulerian circuit is a Eulerian path that starts and finishes at the same vertex.

Undirected graph

An undirected graph is Eulerian if and only if:

- All the vertices are connected.
- Each vertex has an even degree. However, when the start and end vertices are different they must have odd degree.

Directed graph

A directed graph is Eulerian if and only if:

- The graph is strongly connected. That is, there is a path between every vertex.
- For each vertex v , the in degree of v (edges coming in) equals the out degree of v (edges going out). Once again, if the start and end vertices are different then the start vertex must have an out degree one greater than its in degree and the end vertex must have an in degree one greater than its out degree.

Constructing Eulerian paths: Fleury's algorithm (for an undirected graph)

If there are any vertices of odd degree, start at one of those. If every vertex has even degree, then start at any vertex. Now at each step, choose an edge whose deletion would not disconnect the graph, add it to the path and then delete it.

Fleury's algorithm

```
1  find_tour(u):
    if all edges have been deleted:
        return true
2  for each edge e from u to v:
3      remove e from graph
4      success ← find_tour(v)
5      if success:
6          push u onto tour
7          return true
8      else:
9          add e back to the graph
10     return false

11 tour ← a stack
12 start ← { a vertex of odd degree  if one exists
            any vertex                if all vertices have even degree
13 find_tour(start)
```

Aside: Hamiltonian paths